BLUE TEAM THREAT ANALYSIS & INCIDENT DOCUMENTATION

Static Malware Analysis Report: PowerShell_Script

Technical Investigation, Deobfuscation Workflow & Defensive Hardening

1. Document Control & Author Metadata

Field	Details	Field	Details
Author	Jonathan Di Genova	**Role**	Junior SOC Analyst & Cybersecurity Specialist
Date	November 12, 2026	**Classification**	TLP:CLEAR — Portfolio & Study Document
Reference Lab	Hack The Box (Sherlocks)	**Challenge**	PowerShell_Script (Malware Analysis — Very Easy)
Primary Artifact	`script.ps1`	**Analysis Tools**	CyberChef, Static Code Inspection, RDP

2. Case Overview & Scope

During security triage of an isolated Windows workstation (`C:\Users\LetsDefend\Desktop\ChallengeFile\script.ps1`), a suspicious PowerShell script launcher containing a large obfuscated payload was recovered. Initial inspection revealed command-line parameters designed to suppress console visibility and execute an encoded instruction block.

The scope of this investigation is strictly limited to offline static analysis within a controlled sandbox environment. The primary goals are to dissect the execution flags, safely decode the secondary payload without triggering live network callbacks, identify Command-and-Control (C2) infrastructure behaviors, and establish actionable SIEM and endpoint hardening rules.

3. Personal Investigation Timeline

1. **Sandbox Access & Artifact Acquisition:** Connected to the designated Windows analysis virtual machine via RDP (`xfreerdp`) and located `script.ps1` inside the target challenge directory.
2. **Command-Line Wrapper Triage:** Statically inspected the outer `powershell.exe` invocation string to catalog execution flags (`-NoP`, `-sta`, `-NonI`, `-W Hidden`, `-Enc`) and determine their impact on process stealth and user session interaction.
3. **Multi-Stage Payload Deobfuscation:** Extracted the encoded string following the `-Enc` flag and loaded it into CyberChef. Chained `From Base64` with `Decode text` configured to `UTF-16LE (1200)` to remove null-byte spacing and reconstruct readable PowerShell code.
4. **Network & Evasion Logic Analysis:** Parsed the cleartext script to document .NET `WebClient` instantiation, HTTP header spoofing, corporate proxy credential inheritance, and the remote staging endpoint.

5. **In-Memory Execution Sink Review:** Analyzed the final stage of the script, identifying a character-level XOR decryption routine applied to the downloaded response prior to fileless execution via `IEX` (`Invoke-Expression`).

4. Key Evidence & Technical Artifacts

Artifact / Indicator	MITRE ATT&CK	Technical Behavior & Forensic Significance
Launcher Flags (`-NoP -sta -NonI`)	T1059.001 PowerShell	Bypasses local user profile loading (`-NoP`), sets Single-Threaded Apartment mode (`-sta`), and disables interactive prompts (`-NonI`) so user input or error dialogues cannot interrupt execution.
Window Concealment (`-W Hidden`)	T1564.003 Hidden Window	Suppresses the PowerShell terminal window upon execution to conceal activity from the logged-in user desktop session.
Encoded Payload (`-Enc` / Base64)	T1027 Obfuscated Files	Delivers the core script as a Base64-encoded UTF-16LE string, evading naive cleartext command-line inspection.
.NET WebClient (`System.Net.WebClient`)	T1105 Ingress Tool Transfer	Instantiates the .NET HTTP client object (obfuscated with mixed-case characters) to perform outbound web requests and retrieve remote payloads.
Header Spoofing (`User-Agent`)	T1071.001 Web Protocols	Overwrites the default PowerShell HTTP header with a legacy Internet Explorer 11 string (Windows 7 WOW64) to blend malicious requests with routine web traffic.
Proxy Traversal (`CredentialCache`)	T1090 Proxy	Queries `DefaultWebProxy` and attaches `DefaultNetworkCredentials` to authenticate seamlessly through enterprise perimeter proxies.
Staging & XOR Loop (`DownloadString` / `IEX`)	T1140 Deobfuscate/Decode	Contacts an external IPv4 host on port `7443` to fetch an `.asp` resource, applies a character-array XOR key in memory, and executes the result filelessly via `IEX`.

5. Lessons Learned & Hardening Recommendations

- **Enable PowerShell Script Block Logging (Event ID 4104):** Configure Windows Group Policy to record the full deobfuscated script block at execution time, neutralizing Base64 encoding, mixed-case randomization, and in-memory XOR obfuscation.
- **Enforce Constrained Language Mode (CLM):** Restrict standard user endpoints to PowerShell CLM via AppLocker or Windows Defender Application Control (WDAC) to block arbitrary instantiation of sensitive .NET classes such as `System.Net.WebClient` and `CredentialCache`.
- **AMSI & Runtime Buffer Inspection:** Ensure Endpoint Detection and Response (EDR) integrations actively scan memory buffers passed into `Invoke-Expression` (`IEX`) to intercept decoded second-stage payloads prior to execution.

- **SIEM Detection Engineering:** Create high-fidelity detection rules alerting on `powershell.exe` process creation events (Sysmon Event ID 1 / Security Event ID 4688) combining `-Enc`, `-W Hidden`, and `-NonI` flags, especially when correlated with outbound connections to bare IP addresses on non-standard ports.
- **Egress Proxy Filtering:** Configure corporate web proxies to block direct HTTP requests to raw IPv4 addresses without domain resolution and flag outdated browser `User-Agent` strings originating from modern Windows workstations.

> **Analyst Reflection:** Dissecting `script.ps1` reinforced the importance of recognizing Windows UTF-16LE encoding nuances during Base64 triage and demonstrated how adversaries chain native .NET proxy and HTTP classes to achieve stealthy, fileless C2 staging.